*Миша Филитов*

# АКОС 6
# IEEE754, SIMD, x87

НИУ ВШЭ ФКН ПМИ 2026

# Демо

# Фиксированная точка

$$2^i + 2^{(i-1)} + \ldots + 2^0 + 2^{-1} + 2^{-2} \ldots .$$

- 2.5 -> 0010.1000
- $3.125 = 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot \frac{1}{2^1} + 0 \cdot \frac{1}{2^2} + 1 \cdot \frac{1}{2^3} + 0 \cdot \frac{1}{3^4} = 0011.0010$

# Фиксированная точка

$$2^i + 2^{(i-1)} + \ldots + 2^0 + 2^{-1} + 2^{-2} \ldots .$$

- 0.2 -> 0000.0011 [00110011] -> 0.1875
- Ошибка составила 0.0125
- Первые 4 биты не используем, а могли бы!

# IEEE754

# Плавающая точка

$$0.0000000000000000000000000000000000016 = 1.6 \cdot 10^{-35} = 1.6\text{E-}35$$

$$(-1)^S \cdot M \cdot 2^E$$

- S - знак
- M - мантисса (значащая часть)
- E - экспонента (степень двойки)

# Нормализация

$$(-1)^S \cdot M \cdot 2^E$$

**Нормализованные значения**

- 1.xxxx * 2^yyyy
- На первом месте всегда 1
- Мы знаем, что там 1 и явно не храним

**Денормализованные значения**

- 0.xxxxx * 2^0000
- Если экспонента состоит из 0, то на первом месте 0
- Нужно для представления близких к 0 чисел

# Особые значения

$$(-1)^S \cdot M \cdot 2^E$$

| Экспонента | Мантисса | Значение |
|---|---|---|
| 11111111 | Все нули | Бесконечность (+ или -) |
| 11111111 | Не все нули | Not a number (NaN) |
| 00000000 | Все нули | Ноль |
| 00000000 | Не все нули | Денормализованный вид |

# Свойства

```c
int main() {
    float nan = NAN;
    float pinf = INFINITY;
    float ninf = -INFINITY;
    float pzero = 0.0f;
    float nzero = -0.0f;

    printf("NaN != NaN:\t%d\n", nan != nan);
    printf("NaN == NaN:\t%d\n", nan == nan);
    printf("NaN < 1.0:\t%d\n", nan < 1.0f);
    printf("NaN > 1.0:\t%d\n", nan > 1.0f);

    printf("+inf > 1e30:\t%d\n", pinf > 1e30f);
    printf("-inf < -1e30:\t%d\n", ninf < -1e30f);

    printf("+0 == -0:\t%d\n", pzero == nzero);

    printf("1 / +0\t\t%f\n", 1.0f / pzero);
    printf("1 / -0\t\t%f\n", 1.0f / nzero);

    printf("0 / 0\t\t%f\n", pzero / pzero);
    printf("inf - inf\t%f\n", pinf - pinf);
    printf("inf / inf\t%f\n", pinf / pinf);
    printf("0 * inf\t\t%f\n", pzero * pinf);

    printf("NaN + 1\t\t%f\n", nan + 1.0f);
    printf("NaN * 5\t\t%f\n", nan * 5.0f);

    printf("isnan(NaN):\t%d\n", isnan(nan));
    printf("isinf(+inf):\t%d\n", isinf(pinf));

    return 0;
}
```

```
NaN != NaN:     1
NaN == NaN:     0
NaN < 1.0:      0
NaN > 1.0:      0
+inf > 1e30:    1
-inf < -1e30:   1
+0 == -0:       1
1 / +0          inf
1 / -0          -inf
0 / 0           -nan
inf - inf       -nan
inf / inf       -nan
0 * inf         -nan
NaN + 1         nan
NaN * 5         nan
isnan(NaN):     1
isinf(+inf):    1
```

https://godbolt.org/z/xce7qn347

# Плавающая точка

$$(-1)^S \cdot M \cdot 2^E$$

| Format | Sign bits | Exponent bits | Mantissa bits | Exponent bias |
|---|---|---|---|---|
| Float | 1 | 8 | 23(+1) | 127 |
| Double | 1 | 11 | 52(+1) | 1023 |

# Округление

| Значение | 1.40 | 1.60 | 1.50 | 2.50 | -1.50 |
|---|---|---|---|---|---|
| К нулю (towards 0) | 1 | 1 | 1 | 2 | -1 |
| Вниз (–inf) | 1 | 1 | 1 | 2 | -2 |
| Вверх (+inf) | 2 | 2 | 2 | 3 | -1 |
| К ближайшему целому вверх | 1 | 2 | 2 | 3 | -1 |
| **Ближайшее четное** | **1** | **2** | **2** | **2** | **-2** |

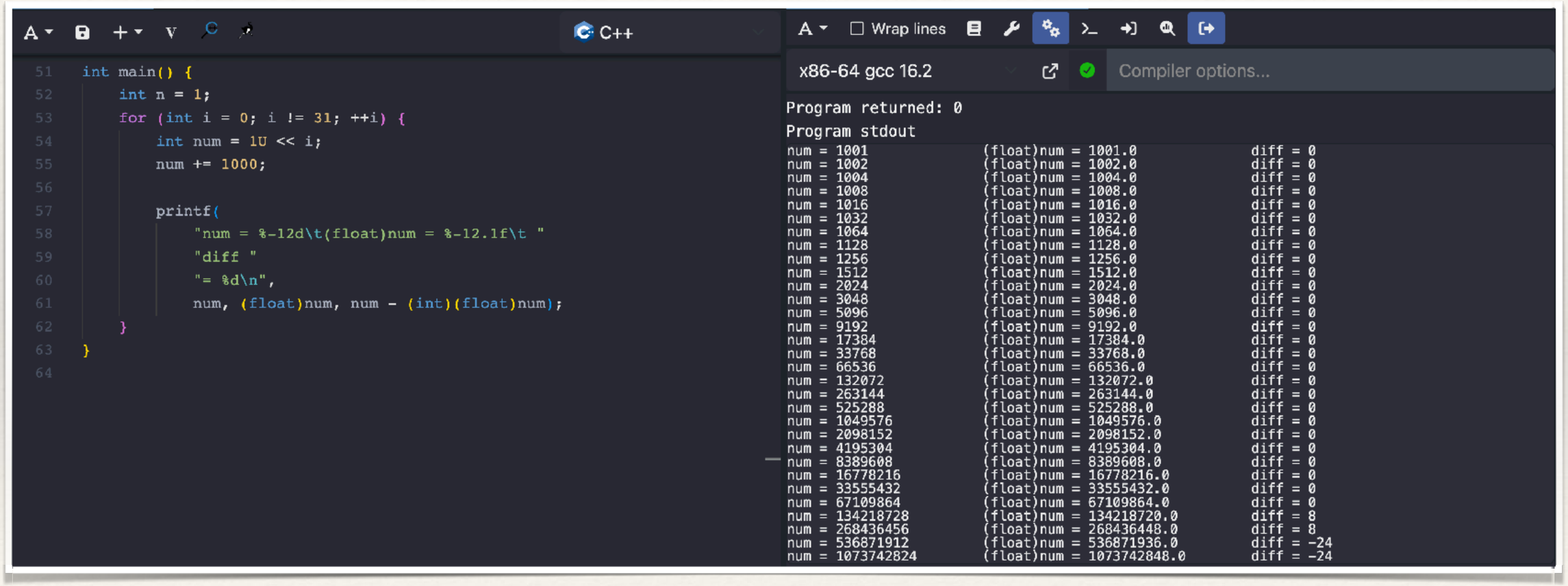
# Округление до 3х знаков

- **1.010**|01 -> 01 меньше половины -> вниз -> 1.010
- **1.010**|11 -> 11 больше половины -> вверх -> 1.011
- **1.010**|10 -> 10 ровно половина -> к четному -> 1.010
- **1.011**|10 -> 10 ровно половина -> к четному -> 1.100

# Min/max

| Число | Значение | Sign | Exponent bits | Mantissa |
|---|---|---|---|---|
| FLT_MAX | $3.402823466 \cdot 10^{38}$ | 0 | 11111110 | 11111111111111111111111 |
| -FLT_MAX | $-3.402823466 \cdot 10^{38}$ | 1 | 11111110 | 11111111111111111111111 |
| FLT_MIN | $1.175494351 \cdot 10^{-38}$ | 0 | 00000001 | 00000000000000000000000 |
| FLT_TRUE_MIN | $1.401298464 \cdot 10^{-45}$ | 0 | 00000000 | 00000000000000000000001 |

# Зачем нам вообще Инты?



```cpp
int main() {
    int n = 1;
    for (int i = 0; i != 31; ++i) {
        int num = 1U << i;
        num += 1000;

        printf(
            "num = %-12d\t(float)num = %-12.1f\t "
            "diff "
            "= %d\n",
            num, (float)num, num - (int)(float)num);
    }
}
```

```
Program returned: 0
Program stdout
num = 1001          (float)num = 1001.0         diff = 0
num = 1002          (float)num = 1002.0         diff = 0
num = 1004          (float)num = 1004.0         diff = 0
num = 1008          (float)num = 1008.0         diff = 0
num = 1016          (float)num = 1016.0         diff = 0
num = 1032          (float)num = 1032.0         diff = 0
num = 1064          (float)num = 1064.0         diff = 0
num = 1128          (float)num = 1128.0         diff = 0
num = 1256          (float)num = 1256.0         diff = 0
num = 1512          (float)num = 1512.0         diff = 0
num = 2024          (float)num = 2024.0         diff = 0
num = 3048          (float)num = 3048.0         diff = 0
num = 5096          (float)num = 5096.0         diff = 0
num = 9192          (float)num = 9192.0         diff = 0
num = 17384         (float)num = 17384.0        diff = 0
num = 33768         (float)num = 33768.0        diff = 0
num = 66536         (float)num = 66536.0        diff = 0
num = 132072        (float)num = 132072.0       diff = 0
num = 263144        (float)num = 263144.0       diff = 0
num = 525288        (float)num = 525288.0       diff = 0
num = 1049576       (float)num = 1049576.0      diff = 0
num = 2098152       (float)num = 2098152.0      diff = 0
num = 4195304       (float)num = 4195304.0      diff = 0
num = 8389608       (float)num = 8389608.0      diff = 0
num = 16778216      (float)num = 16778216.0     diff = 0
num = 33555432      (float)num = 33555432.0     diff = 0
num = 67109864      (float)num = 67109864.0     diff = 0
num = 134218728     (float)num = 134218720.0    diff = 8
num = 268436456     (float)num = 268436448.0    diff = 8
num = 536871912     (float)num = 536871936.0    diff = -24
num = 1073742824    (float)num = 1073742848.0   diff = -24
```

https://godbolt.org/z/qrKsrY5cd

# -114.3

## Получаем бит знака

- Бит знака 1

# -114.3

## Переводим в двоичное представление

```
114 / 2 = 57 [0]
57  / 2 = 28 [1]
28  / 2 = 14 [0]
14  / 2 = 7  [0]
7   / 2 = 3  [1]
3   / 2 = 1  [1]
1   / 2 = 0  [1]
```

```
3 * 2 = 6 [0]
6 * 2 = 2 [1]
2 * 2 = 4 [0]
4 * 2 = 8 [0]
8 * 2 = 6 [1]
Зациклились 1001
```

- 1110010.0100110011001....

# -114.3

## Нормализуем

- Хотим вид 1.xxxxxxx
- Для этого точку надо подвинуть на 6 влево
- 1110010.0100110011001…. = 1.11001001001100110011001…. * 2^6

# -114.3

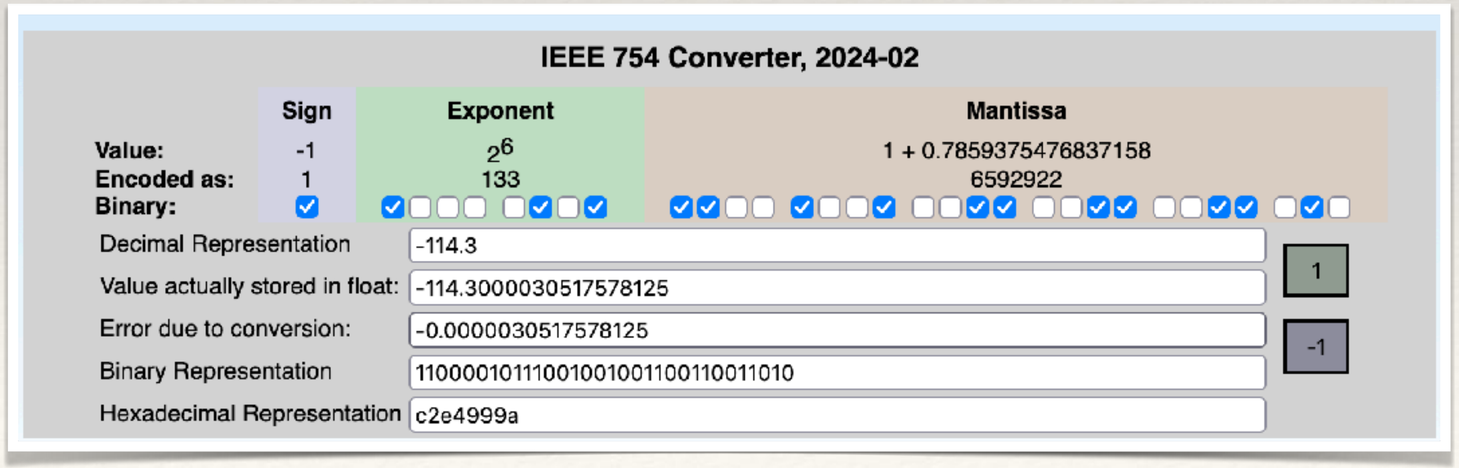
## Округляем

- 1.1100100100110011001 | 1001…. * 2^6
- 1001… > 1000… -> округляем вверх
- 1.1100100100110011010 * 2^6

# -114.3

## 1.11001001001100110011010 * 2^6

- Sign = 1
- Mantissa = 11001001001100110011010
- Exponent = 133 = 10000101




https://www.h-schmidt.net/FloatConverter/IEEE754.html

# Умножение

- $\left((-1)^{S_1} M_1 \cdot 2^{E_1}\right)\left((-1)^{S_2} M_2 \cdot 2^{E_2}\right) = (-1)^{S_1 \oplus S_2}(M_1 M_2) \cdot 2^{E_1+E_2}$
- После умножения надо заново нормализовать и проверить inf

# Сложение

- $(-1)^{S_1} M_1 \cdot 2^{E_1} + (-1)^{S_2} M_2 \cdot 2^{E_2}$
- Пусть $E_1 \geq E_2$:
- Приводим экспоненты: $M_2 \cdot 2^{E_2} = M_2 \cdot 2^{E_2 - E_1} \cdot 2^{E_1}$
- Итоговая мантисса: $M = (-1)^{S_1} M_1 + (-1)^{S_2} M_2 \cdot 2^{E_2 - E_1}$
- $\text{result} = (-1)^S \cdot M \cdot 2^{E_1}$
- Нормализуем и округляем

# Свойства

- Сложение неассоциативно: $(a + b) + c \neq a + (b + c)$

  $(1.0 + 10^{10}) - 10^{10} = 0$ vs $1.0 + (10^{10} - 10^{10}) = 1.0$

- Умножение неассоциативно: $(a \cdot b) \cdot c \neq a \cdot (b \cdot c)$

  $(10^{20} \cdot 10^{20}) \cdot 10^{-20} = \infty$ vs $10^{20} \cdot (10^{20} \cdot 10^{-20}) = 10^{20}$

- Умножение недистрибутивно: $a \cdot (b + c) \neq a \cdot b + a \cdot c$

  $10^{20} \cdot (10^{20} - 10^{20}) = 0$ vs $10^{20} \cdot 10^{20} - 10^{20} \cdot 10^{20} = \text{NaN}$

# Работа с floating point

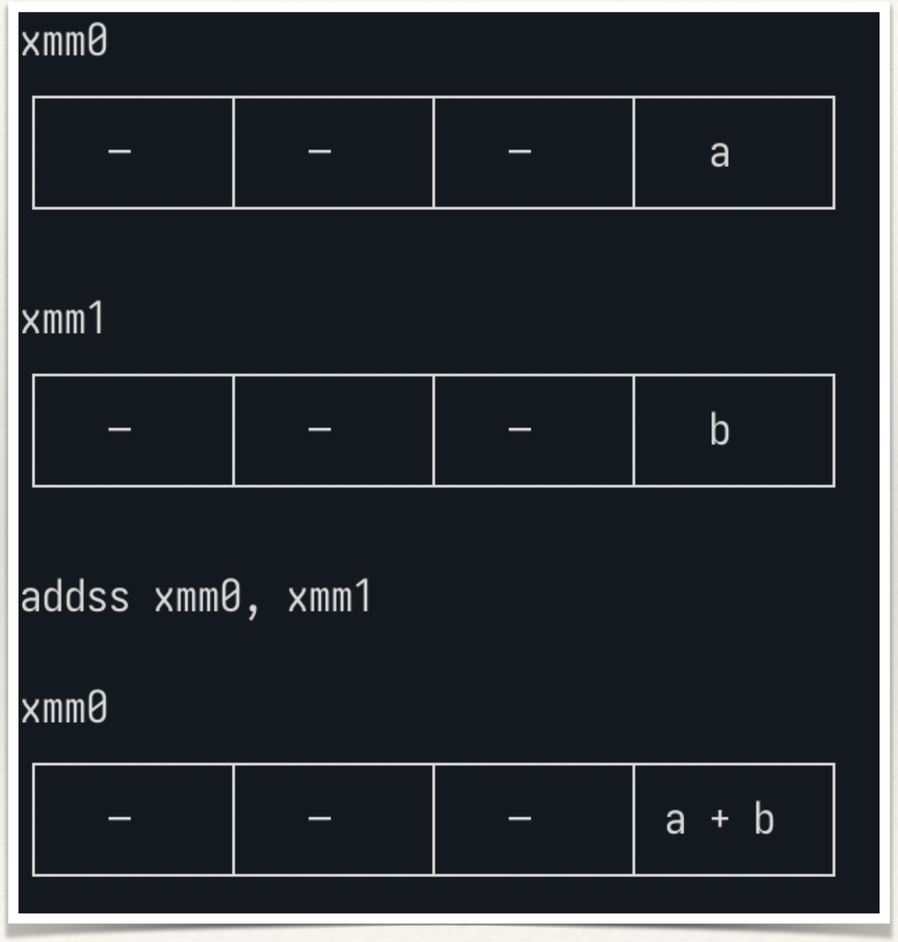
# SIMD - single instruction multiple data

# Широкие регистры

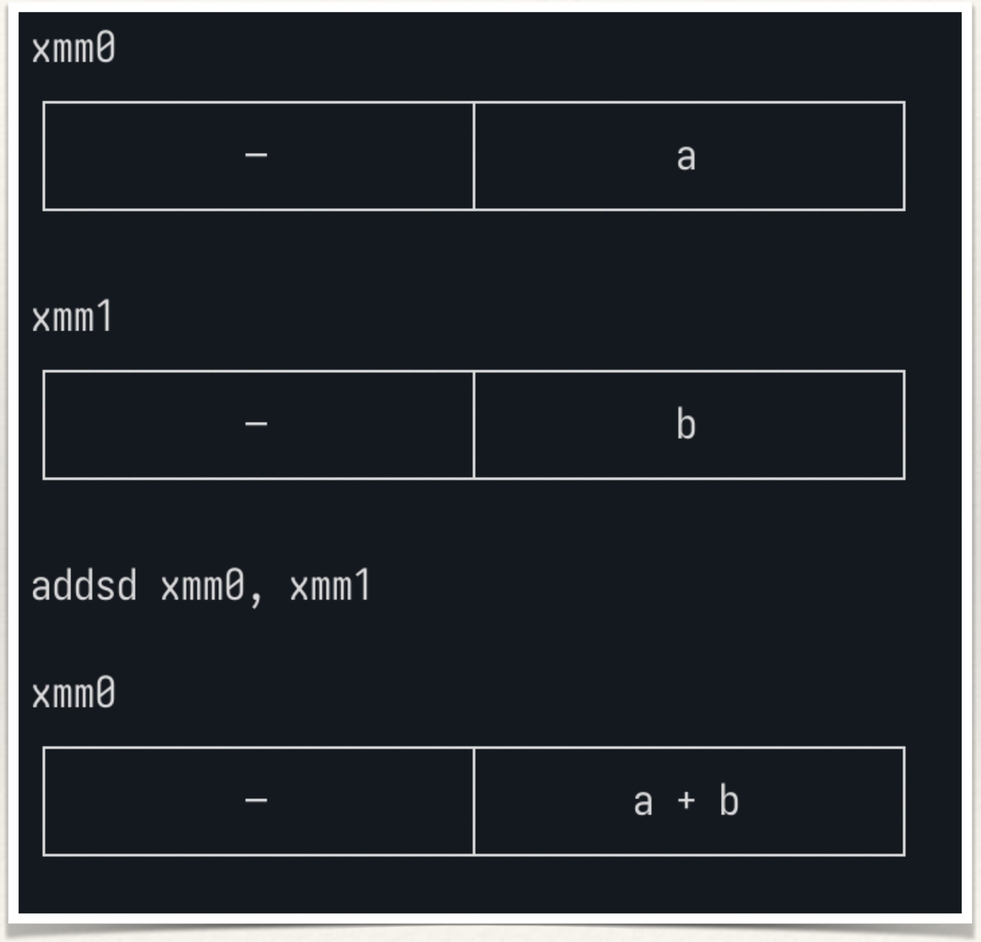
| Набор инструкций | Регистры | Размер регистра | Количество |
|---|---|---|---|
| SSE, x86-32 | xmm0 ... xmm7 | 128 бит | 8 |
| SSE, x86-64 | xmm0 ... xmm15 | 128 бит | 16 |
| AVX / AVX2 | ymm0 ... ymm15 | 256 бит | 16 |
| AVX-512 | zmm0 ... zmm31 | 512 бит | 32 |

xmm - младшие 128 бит ymm
ymm - младшие 256 бит zmm

# Scalar

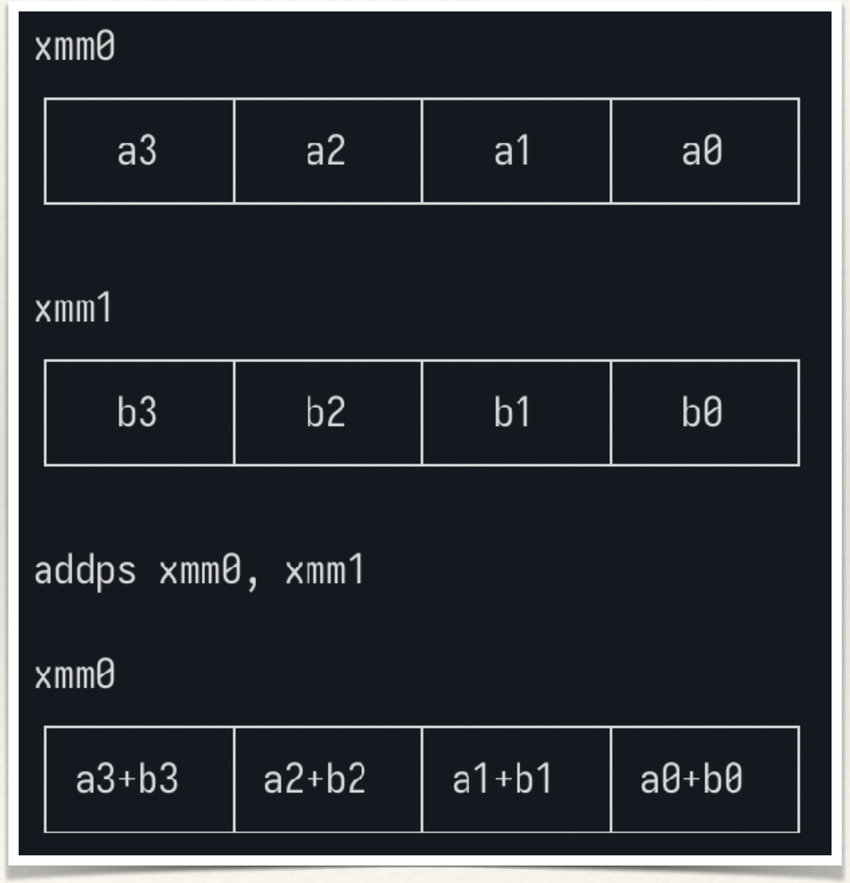



Float

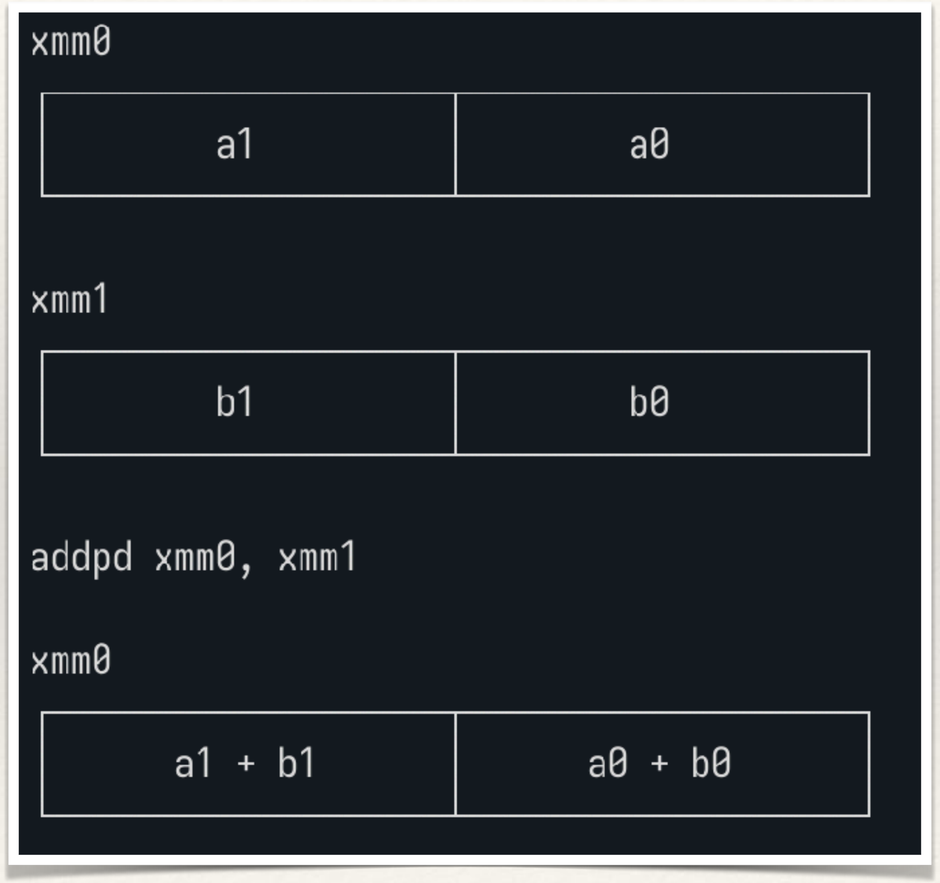



Double

# Packed




Float




Double

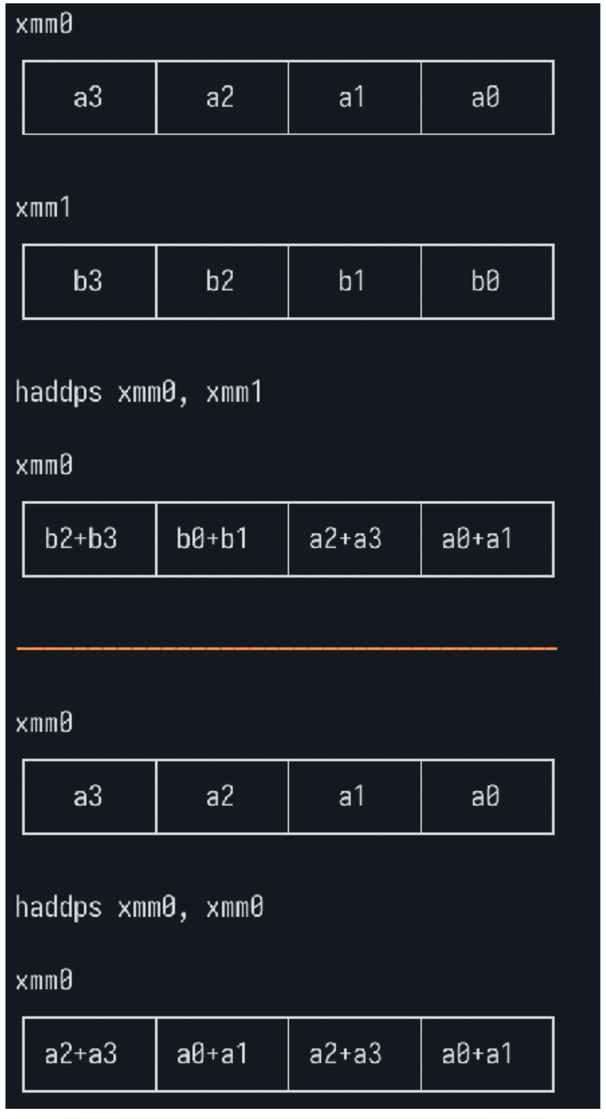
paddb -> 16 × int8

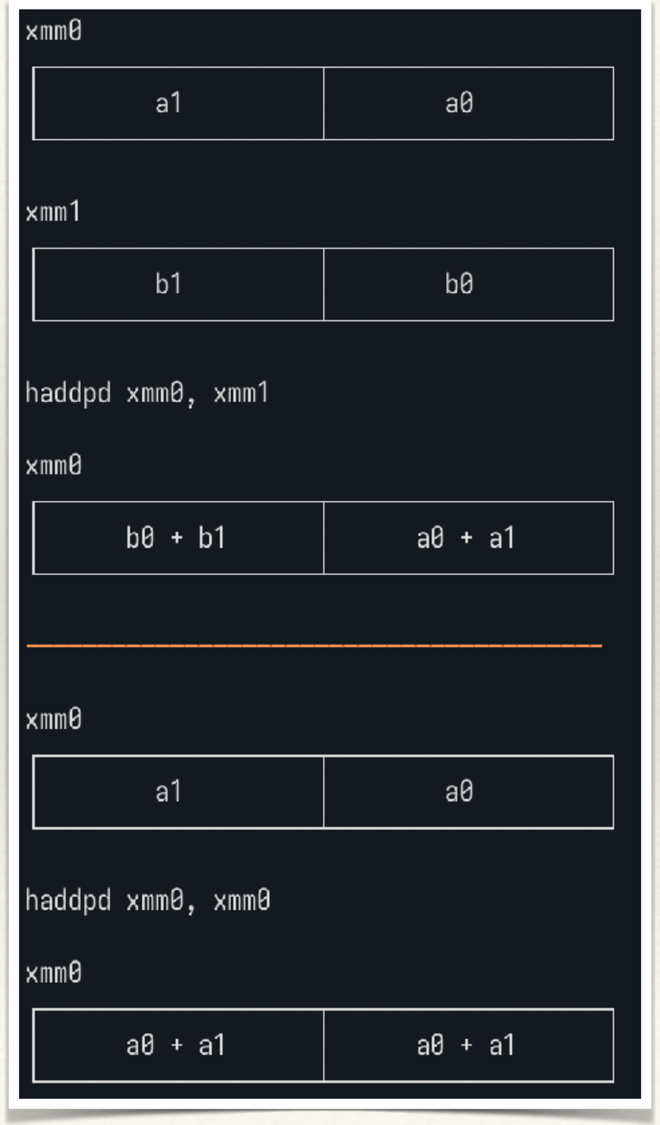
paddw -> 8 × int16

paddd -> 4 × int32

paddq -> 2 × int64

# Горизонтальное сложение



| | |
|---|---|
| VFCMADDCPH | Complex Multiply and Accumulate FP16 Values |
| VFCMADDCSH | Complex Multiply and Accumulate Scalar FP16 Values |
| VFCMULCPH | Complex Multiply FP16 Values |
| VFCMULCSH | Complex Multiply Scalar FP16 Values |
| VFIXUPIMMPD | Fix Up Special Packed Float64 Values |
| VFIXUPIMMPS | Fix Up Special Packed Float32 Values |
| VFIXUPIMMSD | Fix Up Special Scalar Float64 Value |
| VFIXUPIMMSS | Fix Up Special Scalar Float32 Value |
| VFMADD132PD | Fused Multiply-Add of Packed DoublePrecision Floating-Point Values |
| VFMADD132PH | Fused Multiply-Add of Packed FP16 Values |
| VFMADD132PS | Fused Multiply-Add of Packed SinglePrecision Floating-Point Values |
| VFMADD132SD | Fused Multiply-Add of Scalar DoublePrecision Floating-Point Values |
| VFMADD132SH | Fused Multiply-Add of Scalar FP16 Values |
| VFMADD132SS | Fused Multiply-Add of Scalar Single PrecisionFloating-Point Values |
| VFMADD213PD | Fused Multiply-Add of Packed DoublePrecision Floating-Point Values |
| VFMADD213PH | Fused Multiply-Add of Packed FP16 Values |
| VFMADD213PS | Fused Multiply-Add of Packed SinglePrecision Floating-Point Values |
| VFMADD213SD | Fused Multiply-Add of Scalar DoublePrecision Floating-Point Values |
| VFMADD213SH | Fused Multiply-Add of Scalar FP16 Values |
| VFMADD213SS | Fused Multiply-Add of Scalar Single PrecisionFloating-Point Values |
| VFMADD231PD | Fused Multiply-Add of Packed DoublePrecision Floating-Point Values |
| VFMADD231PH | Fused Multiply-Add of Packed FP16 Values |
| VFMADD231PS | Fused Multiply-Add of Packed SinglePrecision Floating-Point Values |
| VFMADD231SD | Fused Multiply-Add of Scalar DoublePrecision Floating-Point Values |
| VFMADD231SH | Fused Multiply-Add of Scalar FP16 Values |
| VFMADD231SS | Fused Multiply-Add of Scalar Single PrecisionFloating-Point Values |
| VFMADDCPH | Complex Multiply and Accumulate FP16 Values |
| VFMADDCSH | Complex Multiply and Accumulate Scalar FP16 Values |
| VFMADDRND231PD | Fused Multiply-Add of Packed Double-Precision Floating-Point Valueswith rounding control |
| VFMADDSUB132PD | Fused Multiply-AlternatingAdd/Subtract of Packed Double Precision Floating-Point Values |
| VFMADDSUB132PH | Fused Multiply-AlternatingAdd/Subtract of Packed FP16 Values |
| VFMADDSUB132PS | Fused Multiply-AlternatingAdd/Subtract of Packed Single Precision Floating-Point Values |
| VFMADDSUB213PD | Fused Multiply-AlternatingAdd/Subtract of Packed Double Precision Floating-Point Values |
| VFMADDSUB213PH | Fused Multiply-AlternatingAdd/Subtract of Packed FP16 Values |
| VFMADDSUB213PS | Fused Multiply-AlternatingAdd/Subtract of Packed Single Precision Floating-Point Values |
| VFMADDSUB231PD | Fused Multiply-AlternatingAdd/Subtract of Packed Double Precision Floating-Point Values |
| VFMADDSUB231PH | Fused Multiply-AlternatingAdd/Subtract of Packed FP16 Values |
| VFMADDSUB231PS | Fused Multiply-AlternatingAdd/Subtract of Packed Single Precision Floating-Point Values |
| VFMSUB132PD | Fused Multiply-Subtract of Packed DoublePrecision Floating-Point Values |
| VFMSUB132PH | Fused Multiply-Subtract of Packed FP16 Values |

https://www.felixcloutier.com/x86/

# (a + b) / ((float)c + d)

```
.intel_syntax noprefix
.global process

// (a + b) / ((float)c + d)
process:
    push    rbp
    mov     rbp, rsp

    // xmm0 = a, xmm1 = b, rdi = c, xmm2 = d

    // add s - scalar s - single(float)
    addss  xmm0, xmm1                    // xmm0 = a + b

    cvtsi2ss xmm3, rdi                   // xmm3 = (float)c
    addss   xmm2, xmm3                   // xmm2 = (float)c + d

    // Деление (a + b) / (c + float(d))
    divss   xmm0, xmm2

    mov     rsp, rbp
    pop     rbp
    ret

printf_format:
    .string     "%f\n"
```

```
#include <stdio.h>

float process(float a, float b, int c, float d);

int main() {
    float a = 1.1;
    float b = 2.2;
    int c = 3;
    float d = 4.4;

    float res = process(a, b, c, d);

    printf("%f\n", res);
}
```

https://godbolt.org/z/oqGhdbM5G

```asm
    .global process

process:
    push    rbp
    mov     rbp, rsp

    // Загрузка данных x и y в xmm0 и xmm1
    // unaligned
    movupd  xmm0, [rip + x]                 // xmm0 = x[0], x[1]
    movupd  xmm1, [rip + y]                 // xmm1 = y[0], y[1]

    // Вертикальная сумма (addpd)
    // xmm0[0] = xmm0[0] + xmm1[0]
    // xmm0[1] = xmm0[1] + xmm1[1]
    // 4.4, 6.6
    addpd   xmm0, xmm1
    // Горизонтальная сумма (haddpd)
    // xmm0[0] = xmm0[0] + xmm0[1]
    // xmm0[1] = xmm1[0] + xmm1[1]
    // 3.3, 7.7
    // haddpd  xmm0, xmm1


    // movhpd, movlpd - нельзя с регистрами

    // movsd
    // xmm0 [double1|double2]
    movlpd  [rdi], xmm0         // *res1 = xmm0[0]
    movhpd  [rsi], xmm0         // *res2 = xmm0[1]

    mov     rsp, rbp
    pop     rbp
    ret

    .data
x:
    .double 1.1, 2.2
y:
    .double 3.3, 4.4
```

```c
#include <stdio.h>

void process(double*, double*);
int main() {

    double res1, res2;
    process(&res1, &res2);
    printf("%f, %f\n", res1, res2);

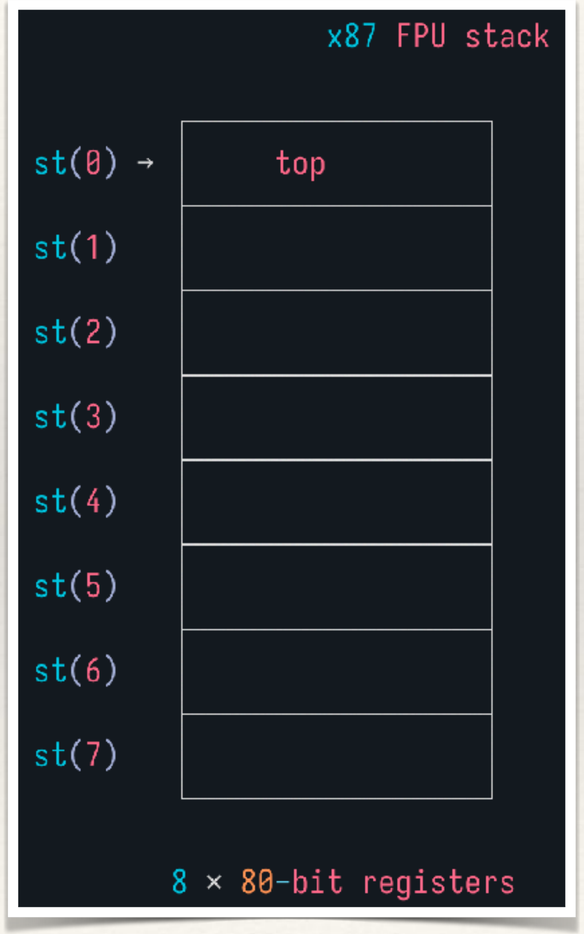
}
```

https://godbolt.org/z/ovP15aa5T

# FPU

# FPU x87

- Отдельный floating-point сопроцессор
- 8 регистров: st(0) ... st(7)
- Каждый регистр - 80 бит
- Регистры организованы как стек
- extended precision: 1 + 15 + 64 бит
- Сейчас в основном legacy / long double



# FPU x87

```
# Загрузка / сохранение

fld   [x]              → push floating-point x в FPU stack
fild  [x]              → push integer x в FPU stack

fst   [x]              → записать st(0) в память
fstp  [x]              → записать st(0) в память и pop

fld   st(i)            → push копии st(i)
fxch  st(i)            → swap(st(0), st(i))

fld1                   → push 1.0
fldz                   → push 0.0
fldpi                  → push pi
```

```
# Арифметика

fadd  st(0), st(i)     → st(0) = st(0) + st(i)
fsub  st(0), st(i)     → st(0) = st(0) - st(i)
fmul  st(0), st(i)     → st(0) = st(0) * st(i)
fdiv  st(0), st(i)     → st(0) = st(0) / st(i)

faddp st(1), st(0)     → st(1) += st(0), pop
fsubp st(1), st(0)     → st(1) -= st(0), pop
fmulp st(1), st(0)     → st(1) *= st(0), pop
fdivp st(1), st(0)     → st(1) /= st(0), pop

fiadd [x]               → st(0) += integer x
fisub [x]               → st(0) -= integer x
fimul [x]               → st(0) *= integer x
fidiv [x]               → st(0) /= integer x

fsqrt                   → st(0) = sqrt(st(0))
fabs                    → st(0) = abs(st(0))
fchs                    → st(0) = -st(0)
```

```
# Сравнения

fucomi  st(0), st(i)    → сравнить st(0) ? st(i), выставить ZF/PF/CF
fucomip st(0), st(i)    → то же + pop

jb    → <
je    → ==
ja    → >
jp    → unordered (NaN)
```

```asm
process:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 32

    // Сохраняем аргументы на обычный стек
    movss   DWORD PTR [rbp - 4],  xmm0    // a
    movss   DWORD PTR [rbp - 8],  xmm1    // b
    mov     QWORD PTR [rbp - 16], rdi     // c
    movss   DWORD PTR [rbp - 20], xmm2    // d

    // st(0) = a
    fld     DWORD PTR [rbp - 4]

    // st(0) = a + b
    fadd    DWORD PTR [rbp - 8]

    // st(0) = c
    // st(1) = a + b
    fild    QWORD PTR [rbp - 16]

    // st(0) = c + d
    // st(1) = a + b
    fadd    DWORD PTR [rbp - 20]

    // st(0) = (a + b) / (c + d)
    fdivp   st(1), st(0)

    // float возвращается через xmm0
    fstp    DWORD PTR [rbp - 24]
    movss   xmm0, DWORD PTR [rbp - 24]

    mov     rsp, rbp
    pop     rbp
    ret
```

process.S 42,1 Bot

```c

float process(float a, float b, int c, float d);

int main() {
    float a = 1.1;
    float b = 2.2;
    int c = 3;
    float d = 4.4;

    float res = process(a, b, c, d);

    printf("%f\n", res);
}
```

main.c

https://godbolt.org/z/5KoxTo1G6